Phase 5 — D2 Concepts and Philosophy

Status: Living derived-design document. Open to D2 Designer-originated completion, clarification, and expansion. Later-phase implementation convenience does not by itself justify revision.

Item 1 — Harness First

D2 should naturally bias toward establishing the strongest practical constraints before expanding the design or implementation space.

Constrain first; construct within the constrained space.

The unconstrained space of possible system designs is extremely large. Even apparently simple choices create a combinatorially large design space. LLM-based construction is especially vulnerable because an LLM can readily continue producing plausible designs or code along many different paths.

Harness is used in a broad D2 sense. It includes any practical semantic, behavioral, evaluative, observational, physical, or supporting constraint that materially narrows the acceptable design space and makes deviation visible. A harness may include representative usage examples, predecessor behavior, rules, standards, tests, evaluation methods, environmental constraints, monitoring requirements, or other mechanisms.

The objective is not merely to test a completed result. The objective is to reduce the freedom of later design and implementation before that freedom produces unnecessary choices. By the time detailed implementation begins, the remaining implementation freedom should be substantially narrower than the original design space.

Predecessor behavior as harness

For an upgrade from a Predecessor D1, the existing system provides an unusually strong source of harness. D2 may derive representative input–output pairs, workflows, behavioral examples, and semantically documented expected behavior. If existing behavior is intended to remain valid in D0, it becomes a constraint on later design.

This is an important reason D2 initially focuses on upgrading an existing D1 rather than designing an unconstrained new system.

Harness before detailed design

Harness First applies recursively throughout D1 design. The preferred conceptual order is:

Establish design generality and intent → establish the harness appropriate to that generality → perform more detailed design within the harness.

A high-level design should therefore be followed, where practical, by identification of the constraints that will govern its descendants before substantial lower-level design proceeds.

Design before implementation

D1 should narrow semantic and structural choices before implementation details are specified. Before an implementation specification becomes highly detailed, D1 should establish how important behavior, assumptions, and failure conditions will be constrained, evaluated, or observed.

Monitoring before usage

Where health and monitoring are material to D0, D1 should design the relevant health status, monitoring behavior, and failure visibility before finalizing the detailed implementation specification of the system being monitored.

The preferred order is:

Define what must remain visible and how system health will be judged → design the detailed behavior that must satisfy those observational requirements → implement.

General philosophy

Harness First is a strong tendency rather than an absolute rule. Exceptions are legitimate, but the hurdle should be high. At each level of design, D2 should actively seek the strongest useful harness that can reasonably be established before allowing the design space to expand downward.

When a useful harness cannot yet be established, the missing harness may be explicitly deferred and kept visible. The purpose is progressively constrained design: later design and implementation should become less arbitrary, more observable, and more difficult to deviate from accepted higher-level intent.

Item 2 — Top-to-Bottom Design

D2 should strongly prefer top-to-bottom design over bottom-up design.

Higher-level intent should constrain lower-level design. Lower-level convenience should not normally redefine higher-level intent.

Top-to-bottom design preserves conceptual stiffness. Higher-level design establishes semantic purpose, governing principles, priorities, and constraints. As design moves downward, the acceptable design space should become progressively narrower.

This is closely related to Harness First. Higher-level design provides part of the harness for lower-level design. If implementation choices routinely revise higher-level concepts, the harness loses authority and design becomes implementation-driven.

Lower-level convenience is weak justification for upward revision

A later phase may discover that an earlier decision is inconvenient, expensive, or difficult to implement. This may justify investigation, but it should not normally justify revision of higher-level design.

If an earlier Designer-oriented requirement establishes an easily editable text format, a later implementation phase should not casually replace it with JSON merely because JSON is easier to implement. The normal response is to find a lower-level design that conforms to the higher-level requirement.

A proposal to revise higher-level intent remains legitimate, but it should face a high hurdle. The burden is to show material defect, inconsistency, infeasibility, serious unintended consequence, or sufficiently strong contrary evidence—not mere inconvenience.

Upward revision is allowed but exceptional

Later work may expose contradiction, impossibility, factual error, serious unintended consequence, a missing higher-level requirement, or evidence strong enough to justify reconsideration. In such cases, upward revision may be proposed.

The farther upward a later-phase proposal attempts to revise the design, the stronger the required justification should normally be.

Designer-originated completion is different from bottom-up revision

Some earlier material is intentionally living or non-exhaustive, including the Living Designer Query Catalog, the Living D2 Glossary, and Phase 5 itself. The D2 Designer may later recognize an omitted philosophy, definition, or query example.

Designer-originated completion or clarification of an intentionally open set is not the same as lower-level design pressure revising higher-level intent. Such completion normally faces a much lower hurdle when it remains consistent with established higher-level design.

General philosophy

Design downward from semantic intent. Treat higher-level design as a harness for lower-level design. Require strong justification when lower-level discoveries attempt to revise higher-level decisions. Preserve lower-hurdle amendment for Designer-originated completion, clarification, and expansion of intentionally open working sets.

Item 3 — Human Position First

D2 should conceptually design work around human positions before designing agents.

If this work were performed by a human organization, what position would perform it, what skills would that position require, what authority would it possess, what information would it receive, and what result would it be responsible for producing?

An agent may later occupy or implement a position. The position should not initially be defined by a particular LLM or agent framework.

Current conceptual examples include the D1 Designer, Design Node Builder, D1 Programmer, D1 Technical Manager, D0 Technical Manager, and D0 Operator. A position is a conceptual responsibility boundary.

Positions before agents

A Design Node Builder should ideally be treated as a person with a relatively narrow relevant skill set. The governing contract, environment, inputs, outputs, harness, and design freedom should be sufficiently clear that the position can perform the work without broad knowledge of the entire D1 project.

Prefer improving the work boundary and handoff over requiring every worker to possess broader intelligence and project context. A poorly bounded task should not automatically be solved by assigning a more capable agent.

Design-to-programming handoff

The ideal target is that the D1 Designer completes the design and produces an implementation specification sufficiently complete that the D1 Programmer can implement it without reconstructing the earlier design process.

Perfect separation may not always be achievable, but continuous Designer–Programmer entanglement should not be assumed as the normal process.

D1 Technical Manager and non-code product maintenance

D1 should conceptually recognize a D1 Technical Manager responsible for technical maintenance and controlled upgrade of the D0 product when the required change does not alter product code or require substantive redesign.

D1 Designer: changes what the product is designed to be.
D1 Programmer: changes product code according to implementation specifications.
D1 Technical Manager: maintains and upgrades the technical product package within the established design without changing product code.
D0 Technical Manager: installs and technically maintains a particular D0 deployment.
D0 Operator: performs routine operation and routine user-level monitoring.

If a default timeout changes from five minutes to ten minutes and the value is an explicitly governed product parameter, the D1 Technical Manager should ideally be able to modify the authorized parameter, run the required validation or regression harness, update release state, repackage, and distribute the product without changing code.

No code change does not mean no harness.

Position existence creates design consequences

The existence of a conceptual position may itself impose requirements on product design. If D1 expects a D1 Technical Manager to maintain product-level technical parameters without programming, D0 should intentionally expose appropriate controls to that position.

The deeper question behind a rule such as “no hard-coded numbers” is:

If this value may legitimately require product-level technical adjustment without redesign or programming, has D1 provided the responsible position with an explicit and governed means to adjust it?

Human Position First does not require every literal value to become configurable. It creates a strong tendency to identify the legitimate controller of a variable decision and provide that position with an appropriate control boundary.

Position-oriented configuration and monitoring

Configuration and monitoring should be designed according to position. D0 Operator controls may include daily spending limits, routine scheduling, collection scope, or approved operating choices. D0 Technical Manager controls may include deployment paths, storage endpoints, service configuration, resource limits, credential integration, and deployment health settings. D1 Technical Manager controls may include product defaults, approved provider defaults, retry policy within accepted ranges, supported feature-policy choices, resource profiles, and release or packaging parameters. D1 Designer controls may include semantic meaning, policy boundaries, algorithmic behavior, product invariants, and supported operating models.

The same underlying event may be represented differently for different positions. The objective is to give each position the information required to discharge its responsibility.

Position hierarchy should reduce unnecessary escalation

Work should be performed at the lowest position possessing the necessary responsibility, information, and authority.

Do not escalate a change to a more expensive or conceptually higher position when the change can be safely and intentionally governed at a lower position.

Do not delegate a change downward when the lower position lacks the authority or conceptual context required to judge its consequences.

General philosophy

D2 should ask whether a reasonably qualified human occupying a defined position could perform the assigned work from the provided boundary, information, authority, and harness. If not, D2 should first suspect the position, contract, handoff, or supporting environment before concluding that a more capable agent is required.

The ideal is not to eliminate intelligence from the worker. It is to avoid using broad intelligence as compensation for poorly designed organizational boundaries.

Agentic implementation may later map one agent to one position, one agent to several positions, several agents to one position, or another execution structure. Conceptual positions remain prior to those implementation choices.

Item 4 — Quality over Expediency

D2 should strongly prefer quality over expediency in D2 design, D1 design, and D0 implementation.

Do not accept avoidable structural, semantic, or implementation weakness merely because the weaker solution is faster or easier to produce.

Time, cost, and complexity remain legitimate constraints. Expediency alone should normally be weak justification for knowingly introducing a lower-quality design.

Simplicity without sacrificing correctness

Prefer simplicity without sacrificing correctness or required accuracy. The objective is not the smallest possible design, but the simplest design that correctly satisfies the governing requirements and harness.

Avoid redundancy

Avoid unnecessary redundancy in design and implementation. Repeated substantial logic may indicate that a common responsibility has not been properly identified and represented. Exact line counts are only rough indicators; semantic duplication is more important.

If multiple implementations independently express what is conceptually the same responsibility, duplication should face a burden of justification.

Redundancy in design

The same principle applies above coding. If many Design Nodes require substantially the same internal behavior, D2 should investigate whether the behavior belongs in a common Design Node model or template. If design processes repeatedly create the same contract, report, evaluation, or handoff, the repetition may indicate a missing common mechanism.

Repeated design structure is evidence of a possible shared concept.

Abstract demonstrated commonality; do not manufacture commonality merely to reduce visible repetition.

Quality rules should be derived where practical

Concrete rules may be derived from Quality over Expediency: avoid hard-coded adjustable values, avoid unnecessary code redundancy, maintain common specifications where commonality is genuine, prefer clear responsibility boundaries, avoid unnecessary design layers, preserve semantic accuracy when simplifying, and favor maintainable structures over local shortcuts.

Quality over Expediency does not replace those rules. It provides part of their conceptual foundation. D2 should ideally be able to explain what quality concern a rule protects.

Expediency remains a legitimate constraint

A more sophisticated design may provide negligible quality improvement while imposing substantial cost, complexity, or delay. In such a case, the simpler or faster design may itself be the higher-quality choice when the system is considered as a whole.

Expediency may participate in a design tradeoff. Expediency should not silently become the default reason for accepting known weakness. Material quality compromises should, where practical, be explicit rather than accidental.

General philosophy

Seek correctness, conceptual clarity, simplicity, maintainability, and coherent reuse before optimizing for immediate construction speed. The purpose is to prevent D2, D1, and D0 from accumulating avoidable structural debt through a sequence of locally convenient decisions.

Item 5 — Modularization

D2 should strongly prefer organizing design and implementation into modules with clear boundaries.

A module is a bounded unit of responsibility that can be understood, worked on, evaluated, and related to the surrounding system primarily through an explicit boundary.

The concept applies across D2, D1, and D0. A module may later appear as a Design Node, design phase or bounded design step, service, subsystem, code module, class or function, wrapped external interaction, test environment, monitoring component, communication mechanism, or another bounded unit of responsibility.

These are different expressions of the same higher-level modularization philosophy.

Prefer bounded work

The preferred questions are: What does this unit receive? What is it responsible for? What constraints govern it? What may it change? What must it produce?

Once these boundaries are sufficiently clear, the internal work of the module should be allowed as much local freedom as is consistent with its governing harness.

Strong boundary; local autonomy.

Modules as conceptual sandboxes

A module may be thought of as a conceptual sandbox. This does not necessarily imply technical isolation. It means that the unit has a defined local working context: responsibility, inputs, governing constraints, applicable rules and standards, resources, permitted actions, expected outputs, and an evaluation or acceptance boundary.

The stronger the local context, the less unnecessary global context the module requires.

Modularization in design

The Design Node is a natural example of design modularization. It represents a bounded design responsibility and should ideally receive a sufficiently clear governing boundary to proceed largely within its local context.

A design module is locally bounded but semantically governed from above.

The Design Tree may later provide one mechanism for organizing these design modules. Its exact architecture remains a later design question.

Modularization in implementation

If D0 repeatedly communicates with an LLM, D1 should consider whether the LLM interaction represents a bounded responsibility. Model invocation, retry behavior, request construction, response handling, usage accounting, and error normalization may belong behind a common module boundary.

The reason is not merely reuse. A clear boundary makes it possible to constrain behavior, apply a common harness, monitor the interaction, change implementation, test independently, assign ownership, and reason about the surrounding system with less internal detail.

Module boundaries should expose the right information

A module should hide internal details that surrounding work does not need while exposing the information required for correct use, governance, monitoring, and evaluation.

The objective is not maximum information hiding. The objective is a boundary that exposes the information necessary for responsibility and hides unnecessary internal complexity.

Relationships to other Phase 5 philosophies

Modularization strengthens Harness First because bounded inputs, outputs, behavior, monitoring, failures, and rules are easier to constrain locally.

It supports Human Position First because a well-defined module resembles a well-defined work assignment.

It supports Quality over Expediency by reducing unnecessary coupling, repeated logic, and propagation of implementation details. However, modularization should not become fragmentation.

When to create a module

Create a module when a responsibility has sufficient conceptual identity that giving it an explicit boundary improves understanding, governance, reuse, evaluation, maintenance, or local autonomy.

There is no universal line count, node size, or number of functions that defines a module. Responsibility and boundary clarity should drive the decision.

General philosophy

Identify coherent responsibilities. Give them explicit boundaries. Provide the context and harness required inside those boundaries. Allow local work to proceed with appropriate autonomy. Relate modules through clear interfaces and governing relationships.

A Design Node, code module, service, and agent are not assumed to be the same technical object. They are different possible expressions of a common higher-level idea:

Bounded responsibility with an explicit relationship to the surrounding system.